Project HAR-kNN
CHE 2026-27

M.Sc. in Informatics and Computing Engineering (M.EIC), 2nd Year

U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

DEI DEPARTAMENTO DE
ENGENHARIA INFORMÁTICA

Outline

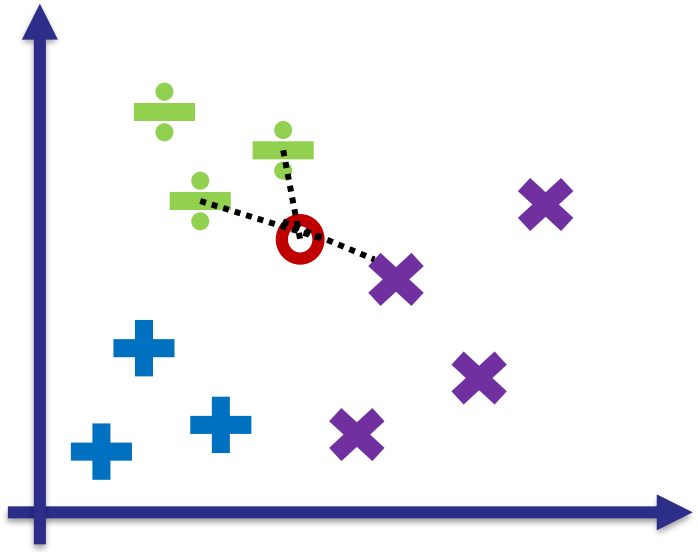
- kNN Machine Learning Algorithm
- kNN for Classification
- kNN Code Provided
- kNN Code Structure
- Project Goals

kNN Machine Learning Algorithm

- Supervised machine learning algorithm
  - Used for regression and for classification
  - Simple and based on the K nearest neighborhoods
  - Scalability problems as it requires the calculation of the distances of the instance to classify to every instance in the training set (knowledge base)
- Lazy training algorithm
  - Training set is stored, and no model is built from the training set
  - => Neglected overhead for online/incremental learning
- There are optimization schemes for kNN that:
  - Represent the knowledge base in data structures (e.g., KD-tree) that make the classification/regression more efficient
  - Provide implementations of approximate kNN (i.e., kNN that may not give results based on the true k NNs)
  - For both see, e.g., Cunningham and Delany, ACM CSUR, 2021)

kNN Algorithm for Classification

- Giving an instance to classify, the algorithm infers/outputs a class for that instance
- Example with vectors with two features (2D space), K=3, 10 instances in the training dataset, and 3 classes:



Instance to classify

Training instance of class A

Training instance of class B

Training instance of class C

Answer: Instance is of class B

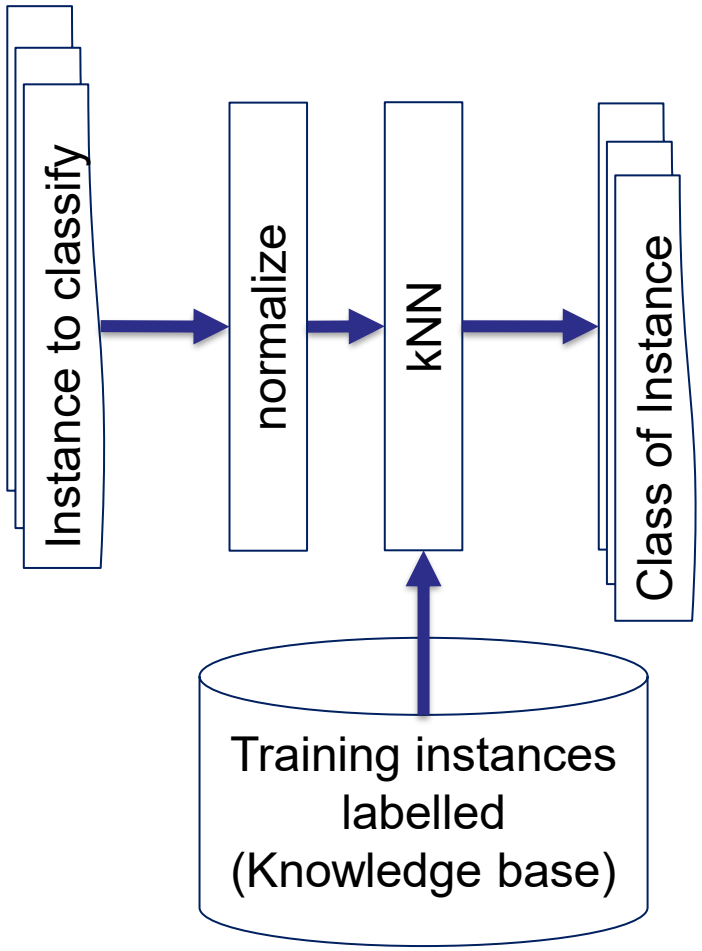
kNN Bibliography

- Source publications:
  - Evelyn Fix and Joseph L Hodges Jr., “Discriminatory analysis-nonparametric discrimination: consistency properties,” Technical report, DTIC Document, 1951. https://apps.dtic.mil/dtic/tr/fulltext/u2/a800276.pdf
  - Thomas M Cover and Peter E Hart, “Nearest neighbor pattern classification,” in IEEE Transactions on Information Theory, 13(1):21-27, 1967. https://doi.org/10.1109/TIT.1967.1053964
- Brief summaries about kNN:
  - “k-nearest neighbors algorithm,” https://en.m.wikipedia.org/wiki/K-nearest_neighbors_algorithm
  - “What is the k-nearest neighbors algorithm?,” IBM, URL: https://www.ibm.com/topics/knn, [accessed in October 2023]
- About kNN
  - Pádraig Cunningham and Sarah Jane Delany. 2021. K-Nearest Neighbour Classifiers - A Tutorial. ACM Comput. Surv. 54, 6, Article 128 (July 2022), 25 pages. https://doi.org/10.1145/3459665
- Relevance of kNN:
  - Wu, X., Kumar, V., Ross Quinlan, J. et al., “Top 10 algorithms in data mining,” in Knowlege Information Systems, 14, 1–37 (2008). https://doi.org/10.1007/s10115-007-0114-2

kNN Code Provided

- C code of a possible implementation of the typical kNN
- Distance calculations use the Euclidean distance by default
- The implementation is targeting a HAR (Human-Activity Recognition) system for embedded devices and provides data for testing
- Code includes a couple of scenarios, including the training and the testing datasets
- The implementation provided does not include the noise reduction and feature extraction stages – inputs are already feature vectors
- The implementation was not tuned to improve the resultant accuracy

kNN Code Structure



- Main includes the outer loop that input the instance to classify to kNN, and kNN returns the inferred class of the instance
- Each instance is represented as a *Point* struct which includes the vector of features
- Prototype of the knn function: *knn_classifyinstance(Point *new_point, K_TYPE k, int num_classes, Point *known_points, int num_points, int num_features)*
- ;

Scenarios and datasets

Benchmark	Type of input	Type of output	input/output sizes
kNN	Sequences of vectors of features	Inferred activity	Small: WISDM Medium: PAMAP2
HAR - kNN	*Sequences of raw data from sensors*	*Inferred activity*	*Small: WISDM* *Medium: PAMAP2*

- WISDM features dataset [WISDM2010] - vectors of features with 43 features
- Features extracted from PAMAP2 dataset [PAMAP2] [PAMAP2012] - vectors of features with 90 features [FMC23]

Scenarios and Datasets (cont.)

Benchmark	K	Dataset	#classes	#features	Training instances	Testing instances
HAR-kNN	3, 5, 20	WISDM	6	43	4336	1082
		PAMAP2	22	90	6186	1008

References

[WISDM2010] Jennifer R. Kwapisz, Gary M. Weiss, and Samuel A. Moore. 2011. Activity recognition using cell phone accelerometers. SIGKDD Explor. Newsl. 12, 2 (December 2010), 74–82. https://doi.org/10.1145/1964897.1964918

PAMAP2012] A. Reiss and D. Stricker, "Introducing a New Benchmarked Dataset for Activity Monitoring," 2012 16th International Symposium on Wearable Computers, Newcastle, UK, 2012, pp. 108-109, doi: 10.1109/ISWC.2012.13.

[PAMAP2] "PAMAP2 Physical Activity Monitoring," https://archive.ics.uci.edu/dataset/231/pamap2+physical+activity+monitoring

[FMC23] Paulo J.S. Ferreira, Joao Mendes Moreira, and Joao M.P. Cardoso. 2023. A Study on Hyperparameters Configurations for an Efficient Human Activity Recognition System. In Proceedings of the 8th international Workshop on Sensor-Based Activity Recognition and Artificial Intelligence (iWOAR '23). Association for Computing Machinery, New York, NY, USA, Article 11, 1–6. https://doi.org/10.1145/3615834.3615851

HAR datasets used

WISDM

- HAR hyperparameters: number of samples per sensor reading = 3
- HAR: number of raw data training instances = 738400
- HAR: number of raw data testing instances = 316400
- HAR: number of features = 43
- HAR: number of classes = 6
- HAR hyperparameters: window size = 200 samples
- HAR hyperparameters: overlap = 50.00 %, 100 instance(s)
- kNN: k = 3
- kNN: number of training instances = 7383
- kNN: number of testing instances = 3163
- kNN: data type used = double
- kNN: number of classified instances = 3163
- kNN: number of classifications wrong = 789
- kNN: number of classifications right = 2374
- kNN: accuracy = 75.06 %

PAMAP2

- HAR hyperparameters: number of samples per sensor reading = 27
- HAR: number of raw data training instances = 1670250
- HAR: number of raw data testing instances = 272190
- HAR: number of features = 90
- HAR: number of classes = 22
- HAR hyperparameters: window size = 300 samples
- HAR hyperparameters: overlap = 10.00 %, 30 instance(s)
- kNN: k = 3
- kNN: number of training instances = 6186
- kNN: number of testing instances = 1008
- kNN: data type used = double
- kNN: number of classified instances = 1008
- kNN: number of classifications wrong = 235
- kNN: number of classifications right = 773
- kNN: accuracy = 76.69 %

HAR-KNN JACA Benchmark Suite

Benchmark structure

- JACABENCH/har-knn:

```
├──knn-ver1.0a
│   ├──bin
│   ├──build
│   ├──run
│   └──src
├──scenario-pamap2
└──scenario-wisdm
```

Benchmark source code

- C source code

```
└──src
    features.c
    features.h
    io.c
    io.h
    knn.c
    knn.h
    main.c
    params.h
    timer.c
    timer.h
    types.h
    utils.c
    utils.h
```

Source code: parameters

Scenario options: changed when selecting scenario

- #define SCENARIO_FEATURES 1 or 2
- #define NUM_FEATURES 43 or 90

Other options: not to be used

- #define DIMEM 0
- #define VERIFY 0
- #define STREAMING 1
- #define TIMING 0
- #define NORMALIZE 1

Some code variant options: to be explored

- #define DT 2 or 1
- #define READ 4 or 2
- #define SPECIALIZED 1 or 0
- #define MATH_TYPE 1
- #define DIST_METHOD 1
- #define USE_SQRT 1 or 0
- #define K 3

Main source code parameters

- TIMING 0 or *1* // 0: w/o timing measurements; 1: w/ timing measurements; default: 1
- ACCURACY 0 or *1* // 0: w/o calculating and reporting; 1: w/ calculation and reporting; default: 1
- **K** = *3*, 5, 7 or 20 // any value >= 1 but these 4 were the ones considered for the scenarios; default: 3
- **DATA_TYPE** = float or double // default: double
- DIMEM 0 or *1* // 0: w/o dynamic memory allocation; 1: w/ dynamic memory allocation; default: 0
- STREAMING 0 or *1* // 0: w/o considering a possible streaming scheme; 1: considering streaming; default: 1
- **SPECIALIZED** 0 or 1 // 0: w/o specialization based on K value; 1: use specialized functions according to, for now, K=3; default: 1
- **MATH_TYPE** 0 or 1 // // 0: use the math.h functions in the code; 1: use the math.h functions according to the type float or double, e.g., sqrt or sqrtf; default: 1
- DIST_METHOD 0 or 1 // 1: Euclidean distance; 2: Manhattan distance; default: 1
- **USE_SQRT** 0 or 1 // 0: w/o sqrt in Enclidean distance; 1: w/ sqrt in Euclidean distance; default: 1

Source code: parameters

Scenario options: changed when selecting scenario

- #define SCENARIO_FEATURES 1 or 2
- #define NUM_FEATURES 43 or 90

Other options: not to be used

- #define DIMEM 0
- #define VERIFY 0
- #define STREAMING 1
- #define TIMMING 0
- #define NORMALIZE 1

Some code variant options: to be explored

- #define DT 2 or 1
- #define READ 4 or 2
- #define SPECIALIZED 1 or 0
- #define MATH_TYPE 1
- #define DIST_METHOD 1
- #define USE_SQRT 1 or 0
- #define K 3

Compile Source Code and Execute

- Using make
- JBSPATH: set the JBSPATH environment variable to point to the location of the JacaBenchSuite directory
  - Should be where the "sources" folder is
  - For example:
    - Linux: export JBSPATH=…
    - Windows: set JBSPATH=…
- Goto folder knn (where the Makefile is)
  - Execute make
  - Makefile options = {cean, cleanall, comp, run, comprofiling, runprofiling}
  - If you like to override parameters, such as
    - READ: make run -e READ=1
    - Scenarios/dataset:
      - make run -e READ=1 SCENARIO=pamap2
      - make run -e READ=1 SCENARIO=wisdm (default scenario)

Compile with gcc and w/o makefile

WISDM

- gcc -D K=3 -D DT=1 -D NUM_FEATURES=43 -D SCENARIO_FEATURES=1 -D READ=4 -D TRAIN_PATH=\"..\\scenario-wisdm\\train2.dat\" -D TEST_PATH=\"..\\scenario-wisdm\\test2.dat\" -D DIMEM=0 -D VERIFY=0 -D STREAMING=1 -O2 -Wall -Wextra -std=gnu99 src\knn.c src\utils.c src\timer.c src\io.c src\features.c src\main.c -I..\scenario-wisdm -o harknn -lm

PAMAP2

- gcc -D K=3 -D DT=1 -D NUM_FEATURES=90 -D SCENARIO_FEATURES=2 -D READ=4 -D TRAIN_PATH=\"..\\scenario-pamap2\\train2.dat\" -D TEST_PATH=\"..\\scenario-pamap2\\test2.dat\" -D DIMEM=0 -D VERIFY=0 -D STREAMING=1 -O2 -Wall -Wextra -std=gnu99 src\knn.c src\utils.c src\timer.c src\io.c src\features.c src\main.c -I..\scenario-wisdm -o harknn -lm

READ=3 data embedded in program, arrays statically filled

Outputs w/ k=3

WISDM

- HAR: number of features = 43
- HAR: number of classes = 6
- kNN: k = 3
- kNN: number of training instances = 4336
- kNN: number of testing instances = 1082
- kNN: data type used = double
- kNN: number of classified instances = 1082
- kNN: number of classifications wrong = 340
- kNN: number of classifications right = 742
- kNN: accuracy = 68.58 %

- Total execution time: 0.1333 s
- Inference time: 0.1232 ms
- Throughput: 8118.86 inferences/s

PAMAP2

- HAR: number of features = 90
- HAR: number of classes = 22
- kNN: k = 3
- kNN: number of training instances = 6186
- kNN: number of testing instances = 1008
- kNN: data type used = double
- kNN: number of classified instances = 1008
- kNN: number of classifications wrong = 235
- kNN: number of classifications right = 773
- kNN: accuracy = 76.69 %

- Total execution time: 0.2885 s
- Inference time: 0.2862 ms
- Throughput: 3494.52 inferences/s

Project Goals

- Accelerate the kNN implementation by code optimizations/transformations and by using hardware accelerators
  - It is given an exact kNN implementation – it must be maintained exact!
  - We assume a streaming scenario, i.e., instances to classify arrive sequentially
  - You can change the code as you like, as long as you maintain similar accuracies
- In the end:
  - Report about the work done and analysis of the intermediate and final results
  - Presentation and discussion of the project

In the end

- Let's see which group was able to achieve the fastest kNN and the most energy efficient kNN for each scenario (and each target architecture)

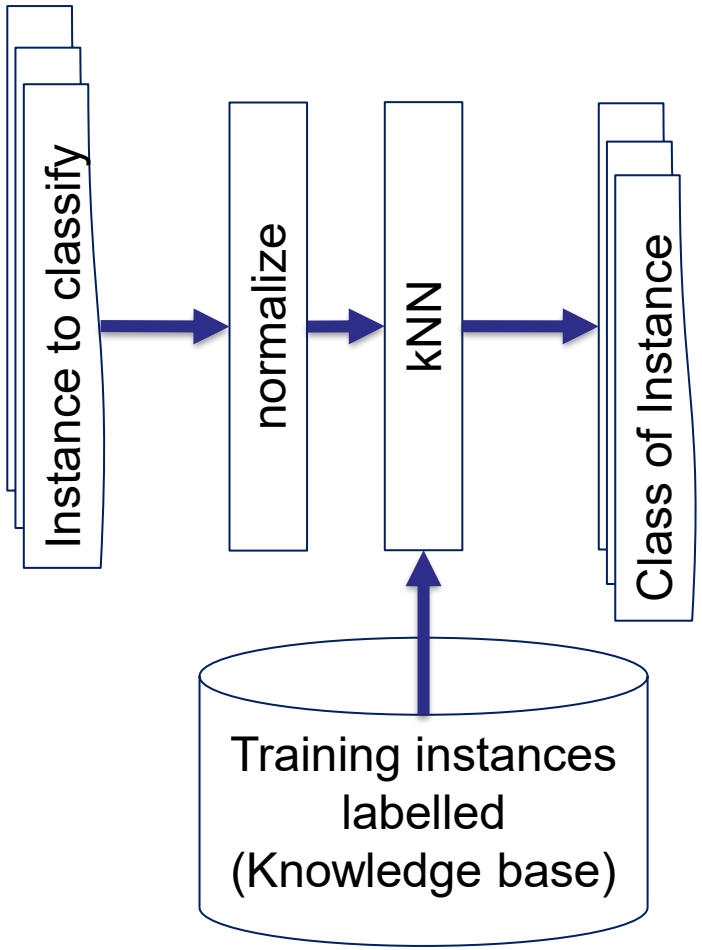
Benchmark	**K**	**Target machine**	**Throughput: #inferences/s**	**Energy efficiency: #inferences/mJ**
kNN - WISDM	3 5 20			
kNN – PAMAP2	3 5 20			

Jaca Benchmark Suite

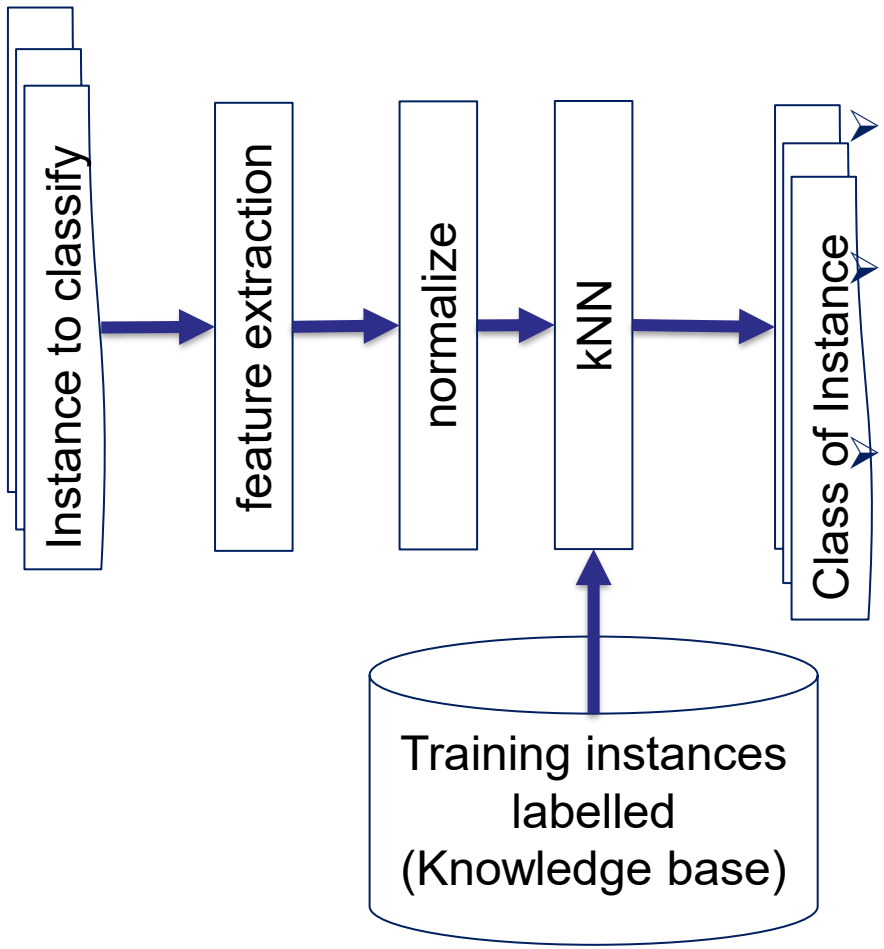
HAR BASED ON KNN

HAR-kNN Code Structure

- kNN considering vectors of features as inputs for inference



- HAR-kNN considering raw data sensors as input



- Main includes the outer loop that input the instance to classify to kNN, and kNN returns the inferred class of the instance
- Main includes the extraction of the features from raw data
- Each instance is represented as a *Point* struct which includes the vector of features
- Prototype of the knn function: *knn_classifyinstance(Point new_point, int k, int num_classes, Point *known_points, int num_points, int num_features);*